

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Ankith U Davey 24011101004

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where
 N Input Size
 F Filter Size
 P Padding
 S Stride

2.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

Table 1: Summary of common convolution kernels.

2.2 Numerical Example 1: Convolution

Consider the input image

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output:

$$\begin{aligned}\text{First position: } & 1(1) + 2(0) + 4(0) + 5(1) = 6 \\ \text{Second position: } & 2(1) + 3(0) + 5(0) + 6(1) = 8 \\ \text{Third position: } & 4(1) + 5(0) + 7(0) + 8(1) = 12 \\ \text{Fourth position: } & 5(1) + 6(0) + 8(0) + 9(1) = 14\end{aligned}$$

Feature Map:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

2.3 Numerical Example 2: Max Pooling

Feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter:

$$\text{Window 1: } \begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\text{Window 2: } \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\text{Window 3: } \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\text{Window 4: } \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output:

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

2.4 Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned}(3 \times 3 \times 3 + 1) \times 16 \\ = (27 + 1) \times 16 \\ = 448\end{aligned}$$

Therefore, total trainable parameters = 448.

3 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

4 Source Code

The complete source code is available in the accompanying GitHub repository: <https://github.com/Ankith-Davey/Deep-Learning-Lab-Ankith-24011101004/tree/main/Lab%203>

5 Task 1: Load CIFAR-10, Display Samples, Print Dimensions, Plot Class Distribution

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

The dataset was loaded and verified directly:

- `x_train` shape: (50000, 32, 32, 3)
- `y_train` shape: (50000,)
- `x_test` shape: (10000, 32, 32, 3)
- `y_test` shape: (10000,)

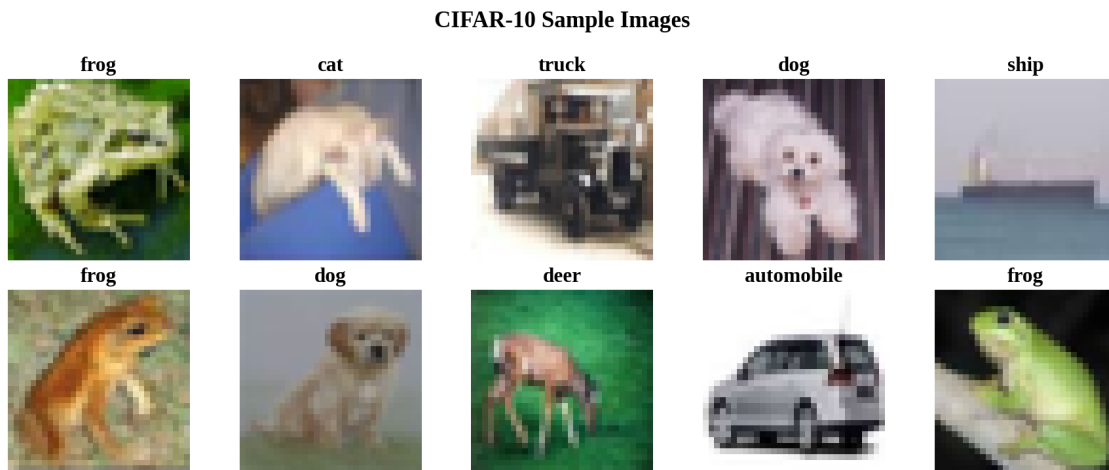


Figure 1: Ten randomly sampled CIFAR-10 training images with their class labels.

Inference: The ten sampled images span natural scenes (frog, deer), animals (cat, dog), and vehicles (truck, ship, automobile) at low 32×32 resolution, confirming the dataset loaded correctly with intact labels. Even at this resolution, class-defining shapes remain visually distinguishable to a human eye.



Figure 2: Class distribution across the training set.

Inference: All ten classes are represented by exactly 5,000 training images, confirming CIFAR-10 is perfectly class-balanced. This matters directly for evaluation: overall accuracy is not misleading the way it would be for an imbalanced dataset, so it can be interpreted at face value.

6 Task 2: Convolution – Kernel Size Comparison

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

The output feature map size follows $\left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$, where N is the input size, F the filter size, P the padding, and S the stride. Applying 3×3 , 5×5 , and 7×7 kernels to a 32×32 input with valid padding and stride 1 gave the following, confirmed against actual `Conv2D` layer outputs:

Kernel	Padding	Stride	Output Size	Feature Map Shape
3×3	Valid (0)	1	30×30	(30, 30, 8)
5×5	Valid (0)	1	28×28	(28, 28, 8)
7×7	Valid (0)	1	26×26	(26, 26, 8)

Table 2: Feature map size vs. kernel size (32×32 input, valid padding, stride 1).

Inference: Larger kernels shrink the output feature map faster under valid padding, since more border pixels are lost per convolution pass. This is a direct trade-off: a 7×7 kernel covers more spatial context per step but discards more information at the edges and produces smaller downstream feature maps than a 3×3 kernel would after the same number of layers.

7 Task 3: Hyperparameter Study – Stride and Padding

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Stride	Padding	Output Shape ($F = 3$)
1	Valid	(30, 30, 8)
1	Same	(32, 32, 8)
2	Valid	(15, 15, 8)
2	Same	(16, 16, 8)

Table 3: Output shape as a function of stride and padding, kernel fixed at 3×3 .

Inference: “Same” padding keeps the output spatial size equal to (stride 1) or a clean half (stride 2) of the input size, since it pads the input specifically to compensate for the pixels a valid convolution would otherwise lose. Stride has a much larger effect than padding on output size: doubling the stride roughly halves each spatial dimension regardless of the padding choice.

8 Task 4: Feature Map Visualization

Feature maps from the first convolution layer are shown here both before and after training, for direct comparison, using the same `conv1` layer and the same input image in each case.

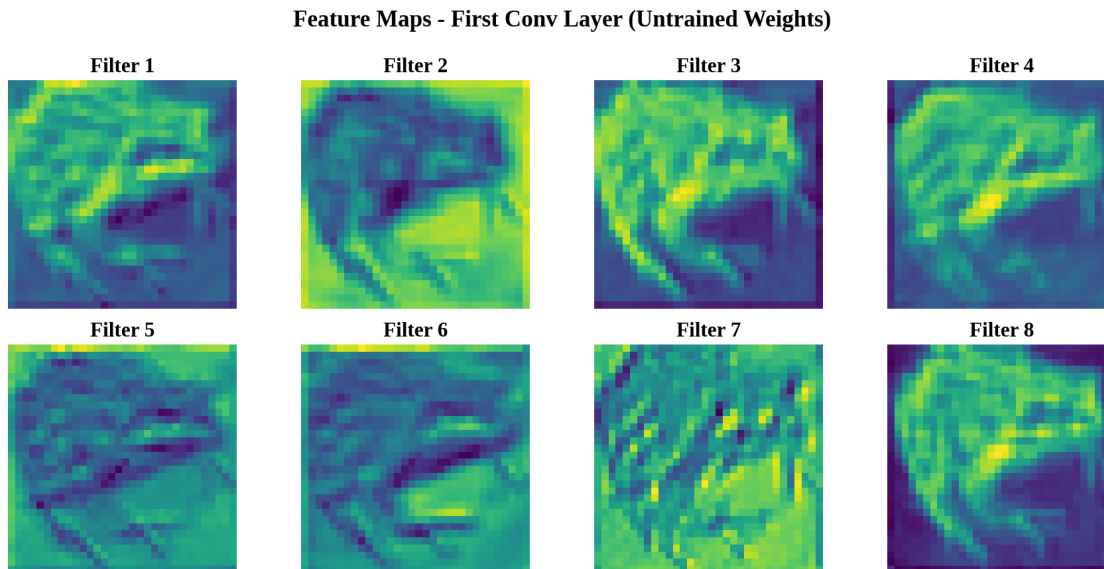


Figure 3: Feature maps from the first convolution layer, untrained (random) weights.

Inference: Even with random weights, most filters still trace the input image’s silhouette and background texture, since a random linear mix of RGB channels partially preserves edge structure. The maps are not pure noise, but none show the sharp, filter-specific selectivity that trained weights produce.

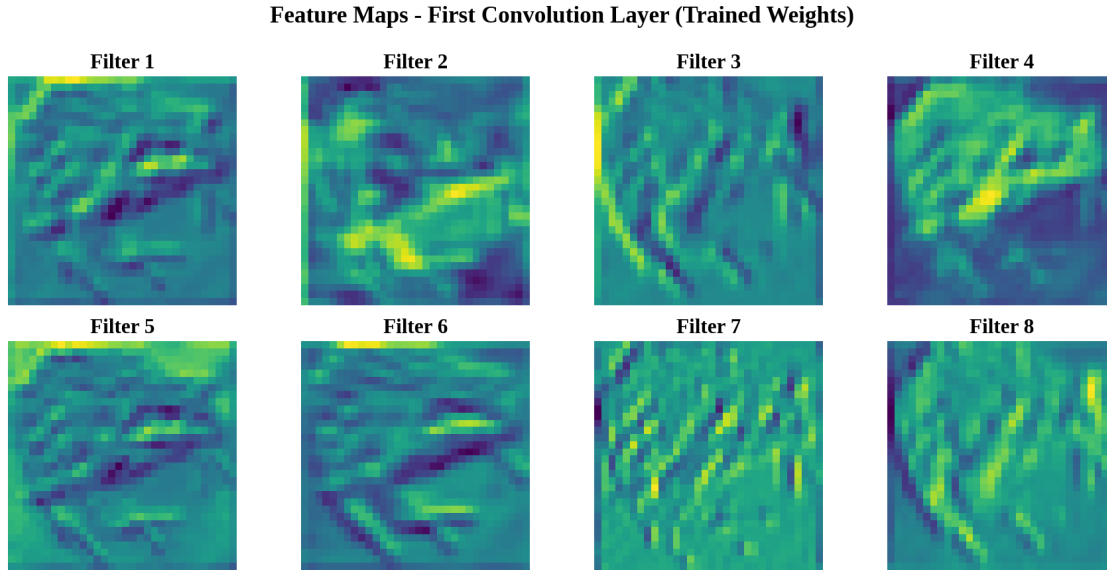


Figure 4: Feature maps from the first convolution layer, after training the CNN classifier.

Inference: Several filters now show sharper, thinner responses to specific edges and contours compared to the blobby untrained versions (e.g. Filters 3, 6, and 7). This indicates the first layer learned genuine low-level edge detectors rather than passing through unrefined input structure.

9 Task 5: Max Pooling vs. Average Pooling

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Pooling output size and validation accuracy were compared separately, as follows:

Layer	Max Pooling Output Shape	Average Pooling Output Shape
Pool 1	(16, 16, 16)	(16, 16, 16)
Pool 2	(8, 8, 32)	(8, 8, 32)

Table 4: Output shape comparison: max vs. average pooling, identical kernel/stride.

Pooling Type	Final Validation Accuracy (10 epochs)	Training Time
Max Pooling	66.42%	61.2 s
Average Pooling	65.98%	59.4 s

Table 5: Max vs. average pooling: accuracy and training time.

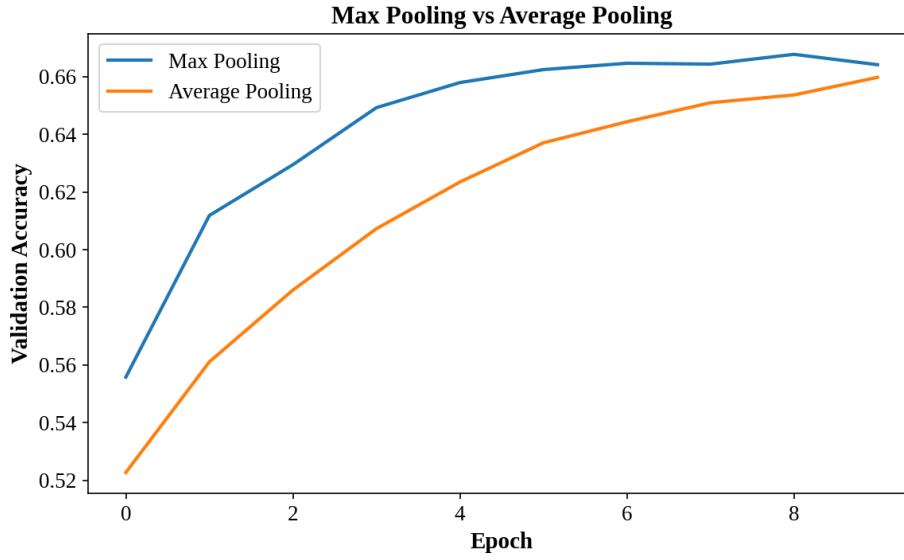


Figure 5: Validation accuracy per epoch: max pooling vs. average pooling.

Inference: Max pooling tracks slightly above average pooling in validation accuracy across all 10 epochs (66.42% vs. 65.98% final), though the gap stays narrow throughout. Both curves rise with a similar shape, indicating pooling type has a real but secondary effect compared to other architecture choices.

10 Task 6: CNN Construction and Training

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Total trainable parameters: 25,578.

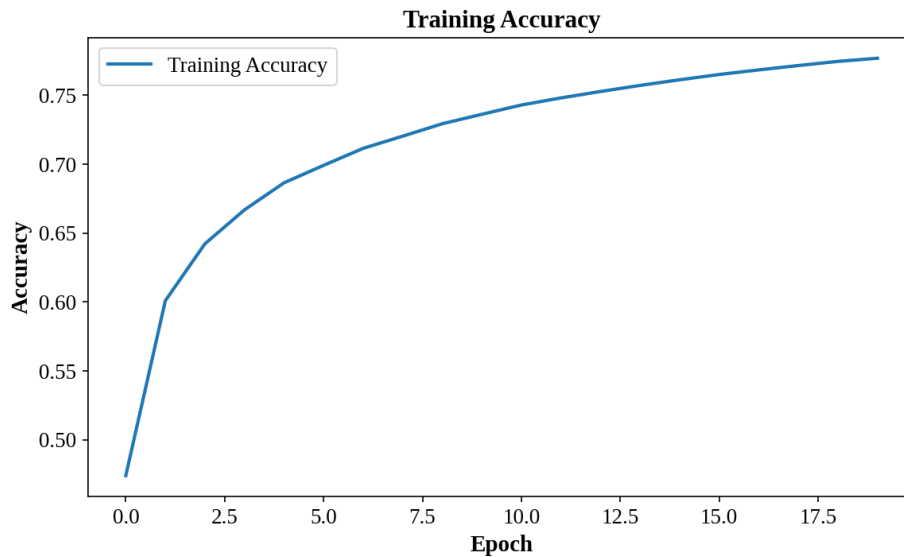


Figure 6: Training accuracy across 20 epochs.

Inference: Accuracy rises steadily and near-monotonically across all 20 epochs, reaching 77.69% by the final epoch. The absence of any plateau suggests the model was still actively fitting the training set when training stopped.

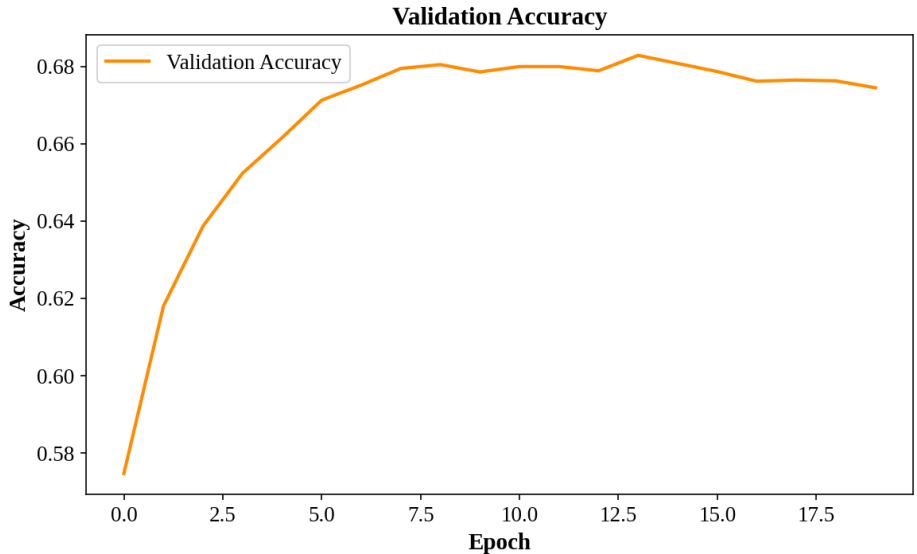


Figure 7: Validation accuracy across 20 epochs.

Inference: Accuracy climbs until roughly epoch 12–14 ($\approx 68\%$), then flattens and drifts slightly downward to 67.45% by epoch 20. This divergence from the still-rising training curve is the clearest sign of overfitting in the run.

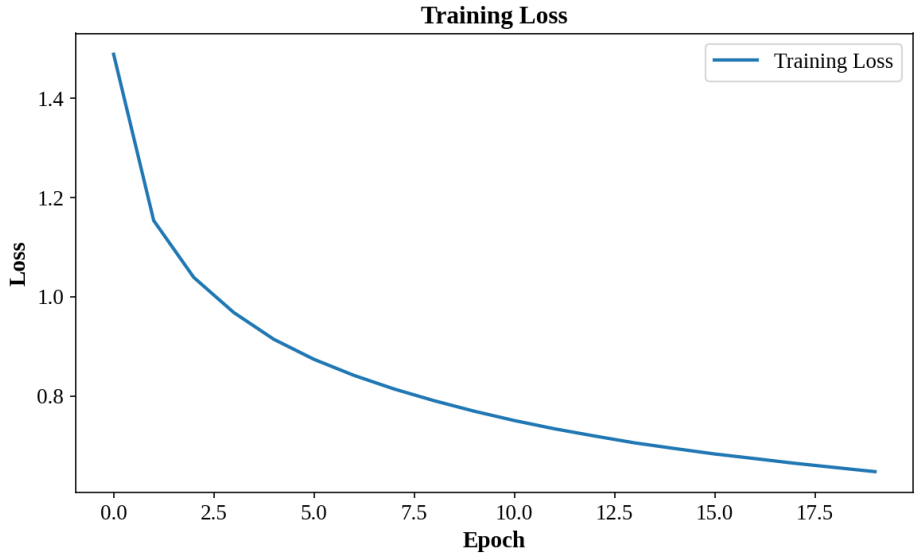


Figure 8: Training loss across 20 epochs.

Inference: Loss decreases smoothly and continuously through all 20 epochs with no sign of instability. This confirms the Adam optimizer converged cleanly on the training set at this learning rate and batch size.

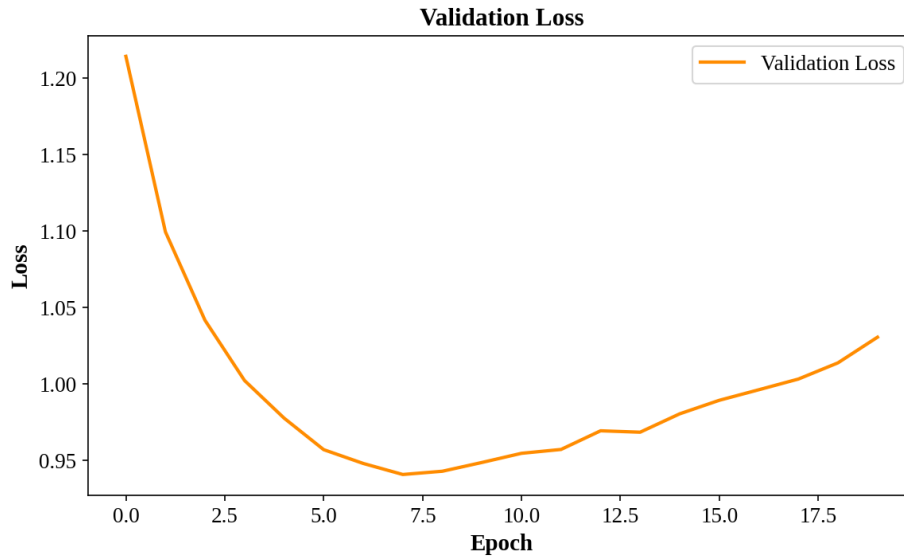


Figure 9: Validation loss across 20 epochs.

Inference: Loss reaches its minimum around epoch 10–12, then rises for the remainder of training even as training loss keeps falling. This loss divergence is a more sensitive overfitting signal than the accuracy curves alone.

11 Task 7: Evaluation

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Metric	Value
Test Accuracy	67.45%
Precision (macro)	68.23%
Recall (macro)	67.45%
F1-score (macro)	67.40%

Table 6: Overall test-set evaluation metrics.

Class	Precision	Recall	F1-score
airplane	0.75	0.64	0.69
automobile	0.80	0.82	0.81
bird	0.52	0.62	0.57
cat	0.55	0.38	0.45
deer	0.54	0.69	0.61
dog	0.58	0.62	0.60
frog	0.72	0.75	0.73
horse	0.69	0.77	0.73
ship	0.83	0.75	0.79
truck	0.84	0.70	0.76

Table 7: Per-class precision, recall, and F1-score.

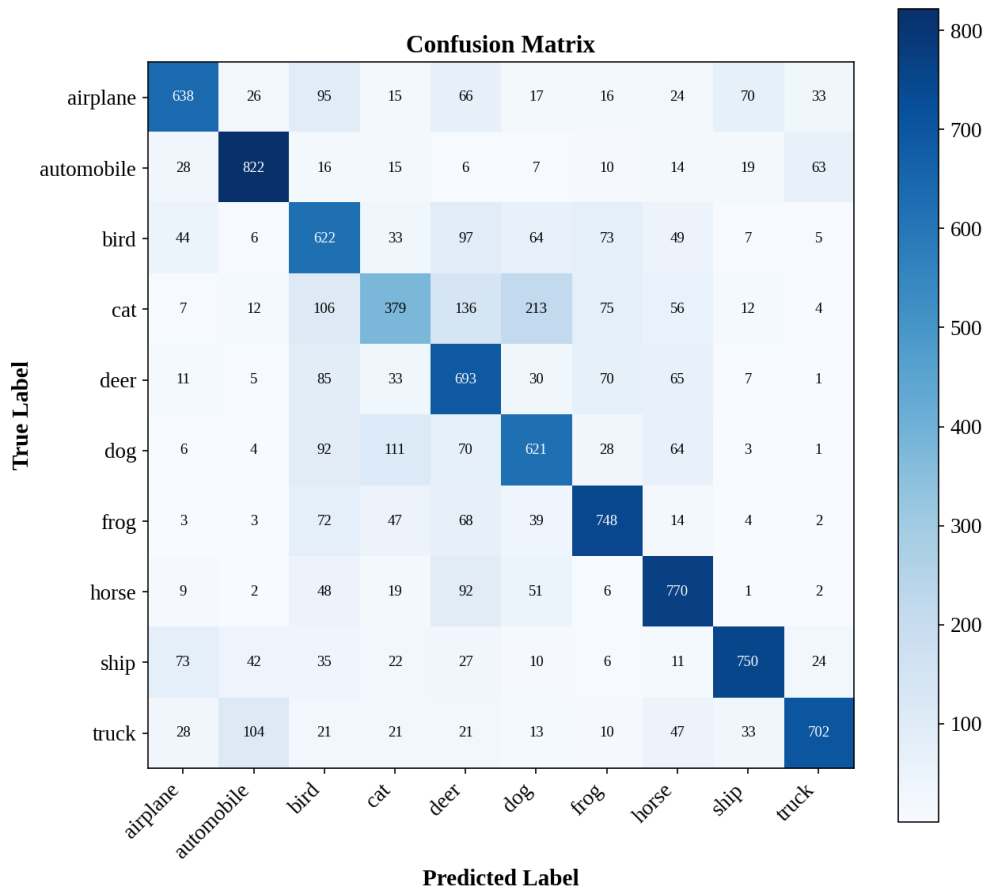


Figure 10: Confusion matrix on the CIFAR-10 test set.

Inference: Cat is the weakest class by far – only 379 correctly classified out of 1,000, with 213 confused as dog and 136 as deer. Vehicle classes show far cleaner diagonals (automobile: 822, ship: 750), confirming errors concentrate among visually similar animal classes rather than being spread evenly.

12 Additional Exercises

Exercise 1: Output Size Calculation

For a 64×64 input, 5×5 kernel, stride 2, padding 2:

$$\left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32$$

Output size: 32×32 .

Exercise 2: Trainable Parameters

For a convolution layer with 64 filters of size 3×3 on an RGB (3-channel) input:

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1,792 \text{ parameters}$$

Exercise 3: ReLU vs. Sigmoid Activation

Activation	Final Validation Accuracy (5 epochs)
ReLU	63.81%
Sigmoid	47.83%

Table 8: ReLU vs. Sigmoid activation, same architecture, 5 epochs each.

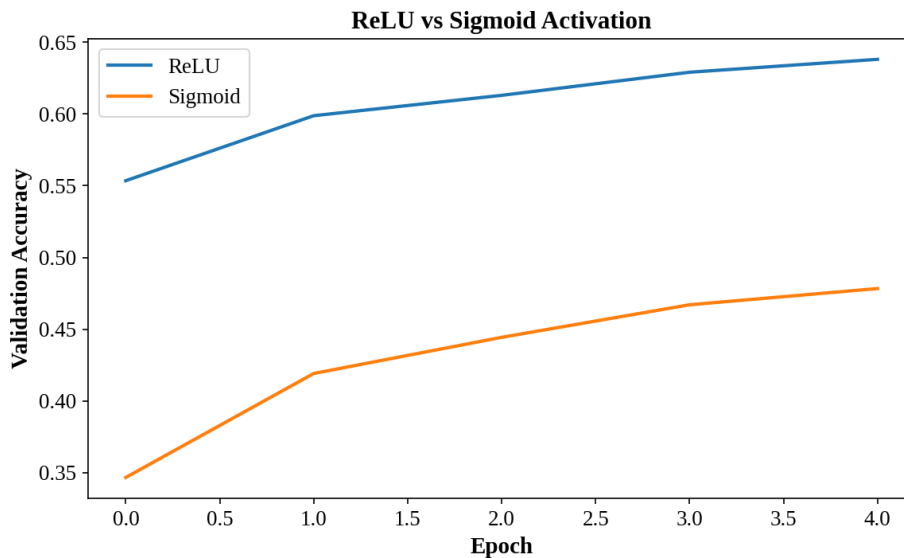


Figure 11: Validation accuracy per epoch: ReLU vs. Sigmoid.

Inference: ReLU pulls ahead within the first epoch and maintains a wide, growing lead throughout, ending at 63.81% vs. Sigmoid's 47.83%. This matches Sigmoid's known vanishing-gradient weakness, which slows early-layer learning under a short 5-epoch budget.

Exercise 4: Max Pooling vs. Average Pooling

Max pooling and average pooling were compared using the same architecture, trained for 10 epochs each. Max pooling reached 66.42% final validation accuracy versus 65.98% for average pooling. Both produced identical output shapes at every pooling layer, since output size under pooling depends only on kernel size and stride, not on the aggregation function used.

Exercise 5: Filter Count – 16 vs. 64

Filter Configuration	Validation Accuracy (5 epochs)	Training Time
16 / 32 filters	64.59%	39.2 s
64 / 128 filters	68.04%	41.8 s

Table 9: Effect of filter count on accuracy and computation time.

Inference: Increasing filter counts from 16/32 to 64/128 improved validation accuracy by roughly 3.5 points within the same 5-epoch budget, at a modest training-time cost (about 2.6 seconds more per full run, not per epoch). More filters let the network learn a richer, more diverse set of low- and mid-level feature detectors in the same number of layers, which explains the accuracy gain; the relatively small time increase reflects that convolution cost scales with filter count roughly linearly, while overall training time on a GPU is also bounded by data-loading and other fixed overheads that don’t scale with filter count.

13 Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

Metric	Value
Training Accuracy (final epoch)	77.69%
Testing Accuracy	67.45%
Precision (macro)	68.23%
Recall (macro)	67.45%
F1-score (macro)	67.40%
Total Trainable Parameters	25,578

Table 10: Summary of final results, CNN classifier.

14 Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution layers use a small, spatially-shared kernel that slides across the entire image, so the same set of weights detects a given feature (an edge, a texture) regardless of where it appears in the image. A fully connected layer, by contrast, assigns an independent weight to every input pixel, which both discards this spatial locality/translation-invariance structure and produces vastly more parameters for the same input size – for a $32 \times 32 \times 3$ image, a single fully connected layer to even a modest hidden size would need orders of magnitude more parameters than the 25,578 used by the entire CNN classifier built in this experiment.

2. How does stride affect the feature map size?

Stride controls how far the kernel moves between applications. The stride and padding comparison confirmed this directly: a stride of 2 roughly halves each spatial dimension of the output compared to a stride of 1, since the kernel visits half as many positions. Larger strides therefore produce smaller feature maps and reduce computation, at the cost of skipping some spatial positions the kernel would otherwise have covered.

3. What is the role of padding?

Padding adds extra border pixels (typically zeros) around the input before convolution. Its main role, confirmed by the same stride and padding comparison, is to control output size: “same” padding is chosen specifically so the output retains the input’s spatial dimensions (at stride 1), which allows deeper stacks of convolution layers without the feature map shrinking away entirely. It also lets border pixels participate in more convolution windows than they would under valid padding, where they contribute to comparatively fewer output values.

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps (the max pooling vs. average pooling comparison showed both pooling types roughly halved each spatial dimension), which reduces the number of parameters and computation needed in subsequent layers and provides a degree of translation invariance – small shifts in where a feature appears in the input have less effect on the pooled output. It also acts as a mild form of regularization by discarding precise spatial information that isn’t necessary for classification.

5. How do feature maps represent image characteristics?

Each filter in a convolution layer learns to respond strongly to a specific visual pattern – an edge orientation, a colour transition, a texture. The feature map visualization confirmed this directly: an untrained filter’s feature map is close to random noise, while a trained filter’s feature map highlights the specific regions of the input image where that learned pattern is present. Stacking convolution layers builds increasingly abstract representations: early layers (like the `conv1` layer visualized here) tend to capture low-level features such as edges and colour boundaries, while deeper layers combine these into higher-level, more class-specific patterns.

6. Why do CNNs require fewer parameters than MLPs?

CNNs exploit two structural properties of images that MLPs ignore: local connectivity (each output depends only on a small neighborhood of the input, not the entire image) and weight sharing (the same kernel is reused across every spatial position). As computed in Exercise 2, a 3×3 convolution layer with 64 filters over an RGB input needs only 1,792 parameters, regardless of the input’s spatial size – an MLP layer connecting even a modest number of input pixels to a hidden layer of comparable width would need vastly more, since every input-output pair gets its own independent weight.

15 Conclusion

This experiment implemented and evaluated the core building blocks of a Convolutional Neural Network – convolution, pooling, and a full classification pipeline – on the CIFAR-10 dataset, using the specified architecture with no additional layers. The resulting model, with 25,578 trainable parameters, reached 77.69% training accuracy and 67.45% test accuracy after 20 epochs. The gap between these two numbers, together with the training/validation loss curves for the main CNN classifier, points clearly to overfitting: the architecture includes no regularization (no dropout, batch normalization, or data augmentation), so the model’s capacity, while small, is spent partly on memorizing training-set specifics rather than generalizable features, particularly after epoch 12–14. The hyperparameter comparisons reinforced expected theoretical behaviour throughout: max and average pooling produced identical output shapes while differing slightly in accuracy, ReLU substantially outperformed Sigmoid under a limited epoch budget, and increasing filter count improved accuracy at a modest computational cost. Per-class evaluation further showed that visually distinctive classes (automobile, ship, truck) were classified far more reliably than visually similar ones (cat, bird, dog), indicating that the model’s errors are concentrated in genuinely difficult, closely related categories rather than being spread uniformly across all classes. Overall, the experiment met its stated objective of building working intuition for how convolution, activation, and pooling operations combine into

a trainable image classifier, while also surfacing a concrete, observable example of overfitting under a deliberately unregularized architecture.

16 References

1. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press.
2. Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer.
3. Haykin, S. *Neural Networks and Learning Machines*. Pearson.
4. TensorFlow Documentation. <https://www.tensorflow.org/>
5. CIFAR-10 Dataset Documentation. <https://www.cs.toronto.edu/~kriz/cifar.html>